

Personal Spotify Listening Tracker V2

Program Specification

Specification version: 1.1

Status: Revised implementation specification - UI/UX design incorporated

Primary purpose: Personal, local-first Spotify listening history, reconciliation, analytics, and visualization system.

1. Product Overview

The Personal Spotify Listening Tracker V2 is a local application designed to maintain a detailed, continuously usable record of Spotify listening activity.

The system combines two fundamentally different types of data:

1. Spotify Extended Streaming History

2. User-requested Spotify data export.

3. Historical and authoritative.

4. Covers the lifetime of the Spotify account.

5. Contains Spotify-recorded playback facts such as exact milliseconds played, start/end reasons, shuffle status, skipped status, offline playback, platform, and private-session information.

6. Live Spotify observations

7. Obtained through permitted Spotify Web API endpoints.

8. Immediate.

9. Useful for displaying current playback and observing information unavailable in Extended Streaming History.

10. May contain information such as active device, current progress, current context, current playback state, shuffle/repeat state, and state-change timestamps.

Spotify currently documents Extended Streaming History as containing lifetime listening records with the stream end time, platform, `ms_played`, track/artist/album identification, Spotify URI, start/end reasons, shuffle, skipped, offline, and private-session information.

Spotify's playback-state API currently exposes active-device information, progress, current item, playback status, context, and a timestamp reflecting playback-state changes such as play, pause, skip, scrub, or track changes.

The application must treat these sources as **complementary evidence**, not competing databases.

The final user-facing result is a single unified listening history.

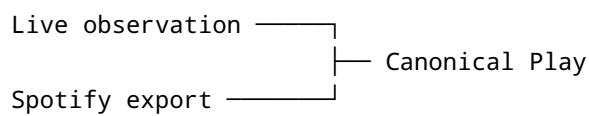
2. Core Philosophy

The program shall follow these principles.

2.1 One real listen = one canonical play

If the live tracker observes a song and a later Spotify export contains the same playback, the user must not see two plays.

Instead:

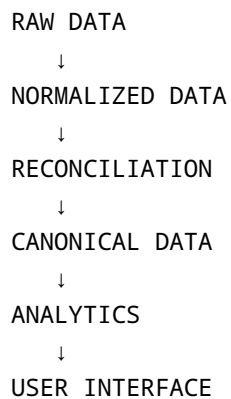


The canonical play represents the program's best understanding of the actual playback.

2.2 Preserve raw evidence

Imported or collected data must never be destructively transformed into the canonical database.

The application shall maintain:



This allows the canonical database to be rebuilt in the future if reconciliation logic improves.

2.3 Known zero is different from unknown

The system must never interpret unavailable information as zero.

Example:

```
pause_count = 0
```

means:

The tracker observed this playback sufficiently and detected no pauses.

Whereas:

```
pause_count = NULL
```

means:

Pause information is unavailable.

This principle applies throughout the application.

3. Primary Goals

The program shall provide:

- lifetime Spotify listening-history imports;
 - up-to-date current playback information;
 - one unified canonical listening timeline;
 - safe repeated imports of overlapping Spotify exports;
 - source-aware reconciliation;
 - detailed track, artist, album, and time-based analytics;
 - historical search;
 - listening-session analysis;
 - clear distinction between confirmed, observed, inferred, and unknown information;
 - local data storage;
 - data export and backup;
 - transparent diagnostics;
 - the ability to rebuild analytics from raw evidence;
 - a configurable feature gate for derived live listening metrics.
-

4. Non-Goals

The application is not intended to:

- replace Spotify as a music player;
- stream Spotify audio itself;
- redistribute Spotify data;
- manipulate Spotify play counts;
- function as a commercial analytics product;
- support multiple unrelated public users;
- become a social network;
- train an AI/ML model on Spotify Content;
- delete raw evidence simply because a more authoritative record becomes available.

The application is primarily a **single-user personal data system**.

5. Technology Stack

Initial implementation should favor simplicity and durability.

Backend

Python

Recommended responsibilities:

- Spotify authentication;
- Spotify API communication;
- import processing;
- reconciliation;
- analytics queries;
- database migrations;
- backup management;
- application API.

A lightweight Python web framework may be used.

Database

SQLite

SQLite is preferred because the application is local and single-user.

Recommended database features:

- foreign keys enabled;
- WAL mode;
- migrations;
- transactional imports;
- appropriate indexes;
- database integrity checks;
- optional FTS indexes for history search.

The architecture must not prevent future migration to PostgreSQL, but PostgreSQL is unnecessary for V1.

Frontend

Local web application.

The UI should work well on:

- desktop;
- laptop;
- tablet-sized browser windows;
- mobile browsers where practical.

The initial target is the user's computer.

6. Project Structure

Recommended conceptual structure:

```
spotify-tracker/  
|  
├─ app/  
|   ├── api/  
|   ├── auth/  
|   ├── database/  
|   ├── importers/  
|   ├── normalization/  
|   ├── reconciliation/  
|   ├── spotify/  
|   ├── tracking/  
|   ├── analytics/  
|   ├── services/  
|   └─ web/
```

```
|
|├── data/
|│   └── tracker.db
|
|├── imports/
|│   ├── pending/
|│   ├── processed/
|│   └── rejected/
|
|├── backups/
|
|├── logs/
|
|├── config/
|
|├── tests/
|
|└── scripts/
```

Exact filenames may differ, but subsystem boundaries should remain clear.

7. Configuration

Configuration must not be scattered through application code.

Example:

```
config/
  settings.toml
```

or environment-based equivalents.

Important settings include:

```
SPOTIFY_CLIENT_ID
SPOTIFY_REDIRECT_URI

USER_TIMEZONE=America/Toronto

ENABLE_DERIVED_LISTENING_METRICS=false

ACTIVE_POLL_INTERVAL_SECONDS=5
```

```
PAUSED_POLL_INTERVAL_SECONDS=15
INACTIVE_POLL_INTERVAL_SECONDS=60

RECENT_SYNC_INTERVAL_SECONDS=60

DATABASE_PATH=data/tracker.db
BACKUP_DIRECTORY=backups/
```

8. Spotify Developer-Policy Feature Gate

8.1 Required flag

The program MUST contain:

```
ENABLE_DERIVED_LISTENING_METRICS=false
```

Default:

```
false
```

Spotify's current Developer Policy explicitly states that developers may not analyze Spotify Content or the Spotify Service to create new or derived listenership metrics, usage statistics, user metrics, benchmarking, profiles, and related analytics.

Therefore this feature must be architecturally separable.

8.2 Flag purpose

This flag primarily controls **metrics or reconstructed historical information derived from Spotify Web API/live playback telemetry**.

It is not merely a UI toggle.

When:

```
ENABLE_DERIVED_LISTENING_METRICS=false
```

the system must not run modules whose purpose is to reconstruct or derive listening analytics from live Spotify API observations.

Live API access may still support direct current-playback functionality such as:

```
Now Playing
Current progress
Current device
Current context
Playing/paused
Shuffle
Repeat
```

provided that functionality otherwise complies with applicable Spotify requirements.

8.3 Disabled behavior

When the flag is `false`, modules such as these must remain disabled:

```
infer_live_play_duration()
infer_skip()
infer_completion()
infer_pause_events()
infer_seek_events()
infer_listening_session()
derive_live_play_record()
calculate_live_usage_metrics()
```

No hidden worker should continue producing those records.

The application may retain whatever limited operational state is necessary to display current playback, but it must not silently turn that transient state into historical derived metrics.

8.4 Enabled behavior

When:

```
ENABLE_DERIVED_LISTENING_METRICS=true
```


the application may activate the implemented derived-telemetry engine.

Potential capabilities include:

- estimated playback duration;
- pause/resume observation;
- seek detection;
- completion estimation;
- inferred skip detection;
- playback transition reconstruction;
- device-transfer observations;
- live listening-session reconstruction;
- additional live-derived analytics.

The program must clearly mark resulting information as:

```
source = live_tracker  
certainty = inferred | observed
```

and never pretend that inferred information came from Spotify Extended Streaming History.

8.5 Important policy disclaimer

Changing this flag to `true` does **not** constitute a determination that the resulting behavior complies with Spotify policy.

The flag is an engineering/compliance boundary.

It exists so that:

- the derived system can remain physically implemented;
- compliant/default operation can disable it;
- experimentation does not require architectural rewrites;
- changes to Spotify policy can be accommodated quickly.

The application should display this warning in advanced settings when enabling the feature.

9. Authentication

Use Spotify OAuth.

Preferred flow:

Authorization Code with PKCE

Reasons include:

- suitable for local applications;
- avoids embedding a client secret;
- supports refreshable authorization.

Tokens must not be hard-coded.

Store authentication data separately from general analytics where practical.

Required Spotify scopes should be kept to the minimum actually needed.

Examples may include:

```
user-read-currently-playing
user-read-playback-state
user-read-recently-played
```

depending on enabled features.

Recently Played currently returns chronological tracks with a `played_at` timestamp and context information and supports up to 50 items per request.

10. Data Architecture

The database should conceptually contain five layers.

Layer 1 — Source files

Information about imported files.

Layer 2 — Raw evidence

Unmodified Spotify export/API payloads.

Layer 3 — Normalized observations

Source-independent structured records.

Layer 4 — Canonical entities and plays

Best known representation of actual listening.

Layer 5 — Analytics

Calculated results and caches.

11. Import Batches

Every imported Spotify package shall create an import batch.

Suggested fields:

```
import_batch
-----
id
source_type
file_name
file_sha256
file_size
requested_at          nullable
imported_at
first_event_at
last_event_at
record_count
new_record_count
duplicate_record_count
error_count
status
application_version
importer_version
notes
```

`source_type` examples:

```
spotify_extended_history
spotify_account_data
manual
legacy_tracker
```

12. Raw Import Records

Every Spotify export row must be preserved.

Suggested:

```
raw_export_event
-----
id
import_batch_id
raw_json
source_fingerprint
ingested_at
```

The program shall not discard unexpected JSON properties.

This is important because Spotify may add fields in later exports.

Unknown properties should remain inside `raw_json` even if the current application does not understand them.

13. Extended Streaming History Importer

The importer must support Spotify Extended Streaming History JSON files.

Spotify currently describes these exports as lifetime records containing 21 categories of information, including playback timing, platform, track identifiers, reasons for playback beginning/ending, shuffle, skip status, offline information, and private-session status.

Expected normalized properties include, where present:

```
ended_at_utc
username
platform
ms_played
country_code
ip_address
user_agent

track_name
artist_name
album_name
```

```
spotify_track_uri

episode_name
show_name
spotify_episode_uri

reason_start
reason_end

shuffle
skipped
offline
offline_timestamp
incognito_mode
```

The importer must tolerate:

- missing fields;
- null fields;
- additional fields;
- different file partitioning;
- multiple JSON files;
- different export generations;
- music;
- podcasts;
- video/content types Spotify may include.

14. Import Deduplication

Repeated Extended History imports must be safe.

The same Spotify event may appear in many exports.

Therefore imported raw events receive deterministic fingerprints.

Conceptually:

```
fingerprint =
HASH(
    event timestamp
    + content URI
    + ms_played
    + platform
    + reason_start
```

```
+ reason_end
+ relevant distinguishing fields
)
```

Fingerprinting must be conservative.

The application must not incorrectly collapse two real consecutive plays merely because they involve the same song.

Possible outcomes:

```
NEW
EXACT_DUPLICATE
POSSIBLE_DUPLICATE
CONFLICT
```

Exact duplicates should not produce additional canonical plays.

15. Spotify Content Entities

Listening events should reference reusable content entities.

Track

```
track
-----
id
spotify_uri
spotify_id
name
duration_ms
explicit
disc_number
track_number
is_local
metadata_updated_at
```

Artist

```
artist
-----
id
spotify_uri
spotify_id
name
```

Tracks must support multiple artists.

Album

```
album
-----
id
spotify_uri
spotify_id
name
album_type
release_date
total_tracks
```

Podcast Episode / Show

The schema should support episodes separately instead of pretending podcasts are music tracks.

16. Canonical Play

The central record is:

```
canonical_play
```

Suggested core fields:

```
id
```

```
content_type
track_id
episode_id

started_at_utc
ended_at_utc

canonical_ms_played

reason_start
reason_end
shuffle
skipped
offline
offline_timestamp
private_session

platform
country_code

reconciliation_status
confidence
created_at
updated_at
```

Some values may be null.

The application must retain field-level provenance instead of relying solely on one overall source field.

17. Field Provenance

Important canonical properties should identify their origin.

Suggested:

```
field_provenance
-----
id
canonical_play_id
field_name
value_source
source_record_id
```



```
certainty
last_updated
```

Possible sources:

```
spotify_extended_history
spotify_account_history
spotify_recently_played
spotify_playback_state
live_tracker
manual
legacy_import
```

Possible certainty levels:

```
authoritative
reported
observed
inferred
estimated
unknown
```

Example:

```
canonical_ms_played
  source = spotify_extended_history
  certainty = authoritative

pause_count
  source = live_tracker
  certainty = observed
```

18. Spotify-Authoritative Precedence

When Extended Streaming History and live observations overlap, Extended Streaming History should normally win for fields Spotify directly records.

Examples:

```
ms_played
skipped
reason_start
reason_end
shuffle
offline
private_session
platform
```

If:

```
Tracker estimate:
181,947 ms

Spotify Extended History:
182,341 ms
```

canonical result:

```
182,341 ms
```

The tracker estimate should remain preserved in raw/normalized evidence for diagnostics.

19. Tracker-Only Information

If derived metrics are enabled, live observations may contain information unavailable from Extended Streaming History.

Examples:

```
pause event
resume event
seek event
observed progress
device name
device ID
device type
device transfer
repeat state
context transition
```

Spotify Extended History must not erase those properties merely because it lacks equivalent fields.

20. Playback Observation

Raw live API polls should be represented separately.

Example:

```

playback_observation
-----
id
observed_at_utc

spotify_state_timestamp

content_uri
progress_ms
is_playing

device_id
device_name
device_type
device_is_active
device_volume_percent

shuffle_state
repeat_state

context_type
context_uri

currently_playing_type

raw_payload

```

This table primarily represents observations, not confirmed plays.

21. Polling

Recommended initial polling strategy:

While actively playing:

5 seconds

Paused:

15 seconds

No active playback:

60 seconds

Recently Played synchronization:

approximately every 60 seconds

Polling should be adaptive and rate-limit aware.

Intervals must be configurable.

22. API Error Handling

Handle:

```
200
204
401
403
429
5xx
network unavailable
Spotify unavailable
```

For 429:

- honor Spotify's retry information;
- apply backoff;
- do not busy-loop.

For authentication expiry:

- refresh token where applicable;
- surface authentication failures clearly if refresh fails.

Network interruptions must not corrupt historical data.

23. Reconciliation Engine

The reconciliation engine links multiple source records to the same real-world play.

Potential match evidence includes:

```
content URI
track identity
timestamps
estimated start time
end time
ms played
neighboring plays
platform
context
ordering
```

Reconciliation must use weighted evidence rather than one brittle condition.

24. Match Classes

Suggested classifications:

```
EXACT_MATCH
HIGH_CONFIDENCE_MATCH
PROBABLE_MATCH
AMBIGUOUS
NO_MATCH
CONFLICT
```

Automatic merging should require sufficiently high confidence.

Ambiguous records should remain unresolved rather than being force-merged.

25. Reconciliation Audit Trail

Each reconciliation decision should be explainable.

Suggested:

```
reconciliation_match
-----
id
canonical_play_id
source_record_type
source_record_id
match_score
match_class
matched_at
algorithm_version
```

Optional match evidence can record:

```
URI match
timestamp difference = 1.2 s
duration difference = 397 ms
same platform
neighboring tracks matched
```

This makes future debugging dramatically easier.

26. Reconciliation Versions

The reconciliation algorithm must have a version number.

Example:

```
reconciliation_algorithm_version = 3
```

If V4 becomes better, raw records can be reprocessed.

Canonical IDs should remain stable where possible.

27. Current vs Finalized Plays

The UI should distinguish recent data that has not yet been reconciled with Extended History.

Conceptually:

PROVISIONAL

means:

known from live/recent sources but not yet confirmed by a later authoritative export.

RECONCILED

means:

matched with authoritative Spotify export evidence.

This does not need to be displayed prominently throughout the normal UI.

An advanced/details screen should expose it.

28. Analytics Engine

Analytics must query canonical plays rather than raw source rows.

This prevents duplicate listening from being counted twice.

Core analytics categories:

Listening time

- total listening time;
- daily;
- weekly;
- monthly;
- yearly;
- lifetime;
- custom ranges.

Plays

- play count;
- full plays;
- partial plays;
- skips;
- repeated plays.

Tracks

- most played;
- most listened;
- first listened;
- last listened;
- recent growth;
- historical rank.

Artists

- total time;
- plays;
- unique tracks;
- first encounter;
- latest listen;
- artist trends.

Albums

- total listening;
- album track breakdown;
- album completion;
- album listening timeline.

Time patterns

- hour of day;
- day of week;
- month;
- year;
- weekday vs weekend.

Devices

Where reliable:

- device/platform usage;
- desktop vs mobile;
- device trends.

29. Metric Eligibility

Not every canonical play can support every statistic.

Example:

A Spotify export from 2023 may have no tracker pause information.

Therefore:

```
pause-rate denominator
```

must include only plays where pause-event coverage is known.

The analytics engine should support:

```
eligible_play_count  
total_play_count  
coverage_percentage
```

Example:

Average pause count: 0.7
Based on 2,841 plays with detailed tracker coverage.

This prevents misleading statistics.

30. Play Counting

The program must explicitly define what qualifies as a play.

There should be separate concepts for:

```
stream event  
meaningful play  
completed play  
Spotify-export row
```

Do not silently equate them.

The UI can provide:

```
Plays
Streams
Listening time
```

where appropriate.

Definitions must remain centralized in the analytics subsystem.

31. Completion Percentage

When sufficient data exists:

```
completion =
canonical_ms_played / track_duration_ms
```

but this metric must account for:

- seeking;
- repeat behavior;
- `ms_played` potentially exceeding nominal duration;
- very short streams;
- restarted tracks.

Clamp or separately model values where necessary.

The raw value should remain available internally.

32. Skip Analysis

Spotify's own Extended History `skipped` property should take precedence for export-confirmed plays.

Tracker-inferred skip status must be distinctly marked.

Analytics should support:

```
Spotify-confirmed skip rate
all-known skip rate
```

if needed.

Unknown skip information must not be treated as `false`.

33. Listening Sessions

A listening session represents a contiguous period of Spotify use.

Session definitions must be configurable.

Example:

```
If gap between canonical plays <= 30 minutes:
    same listening session
else:
    new session
```

Potential session information:

- start;
- end;
- duration;
- tracks;
- artists;
- listening time;
- device/platform;
- dominant artist;
- skips;
- shuffle use.

If session metrics depend on Spotify Platform-derived data, respect `ENABLE_DERIVED_LISTENING_METRICS`.

34. Dashboard

Home

Display:

- Now Playing;
 - today's listening;
 - listening time today;
 - plays today;
 - top artist today;
 - top track today;
 - recent listening;
 - comparison to yesterday;
 - quick links.
-

35. Now Playing

Where available:

Artwork
Track
Artists
Album

Progress
Playing / paused

Device
Context
Shuffle
Repeat

Current playback should be treated separately from finalized historical analytics.

36. History

Provide a chronological playback timeline.

Filters:

```
date
time
track
artist
album
device/platform
shuffle
skip
offline
private session
source coverage
reconciliation status
```

Each play can open a detailed view.

37. Play Detail View

Display only known properties.

Example:

```
Paris
Sabrina Carpenter
Singular: Act I

August 28, 2026
12:41:13 AM - 12:44:16 AM

Listened:
3:02.341

Spotify
Skipped: No
Shuffle: On
Reason started: Previous track finished
Reason ended: Track finished
Platform: iOS

Tracker Details
Pause events: 1
Seek events: 0
Device: iPhone
```

Data
Status: Reconciled
Spotify export: Confirmed
Tracker coverage: Available

If tracker information is unavailable, do not display:

Pause events: 0

Simply omit it or show:

Detailed playback telemetry unavailable

in advanced view.

38. Track Pages

Each track should have its own analytics page.

Potential information:

Total plays
Total listening time
First listen
Latest listen
Average listening time
Average completion
Skip rate
Most active day
Most active month
Listening timeline
Recent trend
Artist
Album

Also show chronological history for that track.

39. Artist Pages

Potential information:

- Total listening time
- Total plays
- Unique tracks heard
- First listen
- Latest listen
- Top tracks
- Top albums
- Listening by month
- Listening by year
- Recent trajectory

40. Album Pages

Potential information:

- Total album listening
- Total plays
- First listen
- Last listen
- Most played tracks
- Least played tracks
- Track-by-track listening
- Album listening timeline

Where technically supportable, eventually detect potential full-album listening sessions.

41. Rankings

Support rankings by:

- plays
- listening time
- average completion
- skip rate
- recent growth

first discovered
most recently heard

Time filters:

Today
Yesterday
7 days
28 days
This month
Last month
This year
Last year
Lifetime
Custom

42. Calendar View

Provide a listening calendar.

Each day may show:

listening time
play count
top artist
top track

Selecting a date opens that day's history.

43. Compare Mode

Support comparisons such as:

Today vs yesterday
This week vs previous week
August vs July
2026 vs 2025

Artist A vs Artist B

Track A vs Track B
Album A vs Album B

Display absolute and percentage differences where meaningful.

44. Search

Global search should support:

track
artist
album
date

Search should tolerate partial names.

Potential later enhancements:

"what did I listen to on August 17?"

should be implemented through structured query features before adding natural-language AI.

45. Discovery Analytics

Potential features:

- newly discovered artists;
- newly discovered tracks;
- rediscovered tracks;
- rediscovered artists;
- tracks not heard for long periods;
- artists increasing rapidly in listening;
- tracks decreasing rapidly in listening.

Definitions must be deterministic and documented.

46. Data Coverage

The application should have a dedicated **Data Coverage** page.

Display:

```
Earliest known play
Latest known play

Extended History coverage
Live tracker coverage
Recently Played coverage

Reconciled plays
Provisional plays
Ambiguous records

Tracker telemetry coverage %
Spotify export coverage %
```

A timeline visualization should make gaps obvious.

47. Import Interface

User should be able to:

1. select/drop Spotify export files;
2. validate them;
3. preview what was discovered;
4. import;
5. view result summary.

Example:

```
Import complete

Files processed: 8
Events found: 82,914
New events: 2,183
Already known: 80,721
Conflicts: 10
```

Canonical plays created: 2,150
Existing plays reconciled: 27
Requires review: 6

48. Import Atomicity

Imports must be transactional.

If processing catastrophically fails halfway through:

```
database state = unchanged
```

A partially completed import must not silently remain.

Large imports may use controlled transactional batches as long as recovery is deterministic.

49. Raw Data Explorer

Advanced users should be able to inspect:

```
raw Spotify event  
normalized record  
canonical record  
reconciliation match  
provenance
```

This is primarily for debugging and transparency.

50. Database Rebuild

The application must eventually provide:

```
Rebuild Canonical Database
```

This operation:

1. preserves raw records;
2. clears derived canonical/reconciliation products as appropriate;
3. reruns normalization;
4. reruns reconciliation;
5. rebuilds analytics.

It must not require re-downloading Spotify history.

51. Database Migrations

Schema changes must be versioned.

Example:

```
schema_version
1
2
3
...
```

Application startup should verify compatibility.

Never alter production database structure through ad-hoc startup SQL without migration tracking.

52. Metadata Enrichment

Spotify catalog information may be used where permitted to associate imported URI-based records with useful content metadata.

Metadata should be cached.

Do not repeatedly request unchanged information.

Each metadata record should contain:

```
last_refreshed_at
```

and potentially:

```
metadata_source
```

The system must continue functioning if Spotify metadata retrieval fails.

Historical listening should never disappear simply because an album lookup cannot be completed.

53. Time Handling

All persisted timestamps should use UTC.

Example:

```
2026-08-28T04:41:13Z
```

Display conversion uses:

```
USER_TIMEZONE
```

Default configuration:

```
America/Toronto
```

DST must be handled through timezone libraries rather than fixed UTC offsets.

Date filters shown in the UI operate in the user's configured timezone.

54. Privacy and Sensitive Fields

Extended History can contain sensitive information such as:

```
IP address  
country  
private-session status  
user agent
```

These should not automatically dominate the normal dashboard.

Suggested treatment:

- retain in raw import;
- optionally normalize;
- hide sensitive technical fields from normal UI;
- expose through advanced record details when useful.

The entire database remains local by default.

55. Logs

Application logs must not unnecessarily contain:

```
OAuth tokens
full Spotify authorization headers
personal IP-history exports
```

Logging levels:

```
ERROR
WARNING
INFO
DEBUG
```

Debug logging should be opt-in.

56. Backups

Provide automatic database backups.

Suggested:

```
backups/
  tracker-2026-08-28.db
```

Recommended behavior:

- before migrations;
- before major rebuilds;

- before destructive maintenance;
- optionally scheduled.

Backups should be validated where practical.

57. Exporting Tracker Data

Allow exporting the program's own canonical data.

Formats:

JSON
CSV

Potential later option:

SQLite snapshot

Export should allow:

- all history;
 - selected date range;
 - selected artist;
 - selected track.
-

58. Maintenance

Provide diagnostics such as:

Database integrity
Schema version
Database size
Raw event count
Canonical play count
Unresolved reconciliation count
Last Spotify export
Last successful API connection
Last backup

59. Performance Requirements

Typical dashboard requests should not scan every raw JSON record.

Frequently queried canonical columns must be indexed.

Likely indexes include combinations involving:

```
ended_at_utc  
started_at_utc  
track_id  
artist relationships  
album_id  
spotify_uri  
reconciliation_status
```

Large historical imports should not freeze the entire UI without progress reporting.

60. Caching

Expensive aggregate results may be cached.

Examples:

```
daily totals  
monthly totals  
track lifetime summaries  
artist lifetime summaries
```

Caches must be reproducible from canonical data.

Never make a cache the only copy of information.

61. Testing

Import tests

Test:

- valid export;
- empty export;
- multiple files;
- duplicate export;
- partially overlapping export;
- null fields;
- unexpected fields;
- malformed JSON;
- podcast records;
- local tracks where possible.

Reconciliation tests

Test:

- obvious match;
- consecutive same-track plays;
- same song played repeatedly;
- tiny timestamp mismatch;
- large timestamp mismatch;
- duration mismatch;
- conflicting sources;
- ambiguous candidates.

Analytics tests

Use deterministic fixture datasets.

Expected totals must be known exactly.

Timezone tests

Include DST boundaries.

Feature-flag tests

Run automated tests with:

```
ENABLE_DERIVED_LISTENING_METRICS=false
```

and:

```
ENABLE_DERIVED_LISTENING_METRICS=true
```

Disabled mode must prove that derived telemetry workers and metric production remain disabled.

62. Startup Sequence

Conceptually:

```
Start
↓
Load configuration
↓
Validate database
↓
Run migrations
↓
Load authentication
↓
Start backend
↓
Start UI
↓
Start permitted Spotify services
↓
If ENABLE_DERIVED_LISTENING_METRICS=true:
    start derived telemetry subsystem
else:
    leave subsystem disabled
```

63. Shutdown

Graceful shutdown should:

- stop pollers;
- finish/rollback open transaction;

- persist safe application state;
- close SQLite cleanly;
- stop local server.

64. Application States

Spotify integration should visibly support states such as:

```
Connected
Disconnected
Authentication expired
Spotify unavailable
Rate limited
No active device
Nothing playing
```

"No active device" is not an application error.

65. Monthly Extended History Workflow

The architecture should support recurring imports without requiring them at a precise cadence.

Typical workflow:

```
Request Spotify Extended History
↓
Continue using Spotify normally
↓
Receive export
↓
Import files
↓
Deduplicate
↓
Normalize
↓
Reconcile existing provisional records
↓
Canonical history becomes more authoritative
```

The application must determine the latest timestamp actually present in the export rather than assuming the export ends on the request date or download date.

66. Data Authority Hierarchy

Default precedence should roughly follow:

1. Spotify Extended Streaming History
authoritative historical playback facts
2. Other explicit Spotify-provided historical facts
where applicable
3. Spotify Recently Played
chronological evidence
4. Live Spotify Playback State
current observation
5. Derived tracker inference
estimated/inferred information

Precedence is **field-specific**.

A lower-ranked source may still contain a field completely absent from higher-ranked sources.

67. No Destructive Superseding

When Spotify later corrects an inferred value:

Do:
mark tracker evidence superseded for canonical use

Do not:
delete tracker evidence

This provides traceability.

68. User Interface Philosophy

The dashboard should feel like a polished personal music analytics application.

It should **not** feel like:

- an SQLite viewer;
- a developer console;
- a raw JSON inspector;
- a Spotify clone.

Normal screens should emphasize:

```
music  
history  
patterns  
statistics  
discovery
```

Technical details belong in advanced sections.

69. Visual Design

Recommended direction:

- modern;
- minimal;
- information-dense without clutter;
- clear typography;
- responsive;
- dark and light mode;
- album artwork where appropriate;
- useful charts rather than decorative charts.

Charts should always have accompanying numeric information.

70. Data Status Indicators

Advanced/details views may show subtle indicators:

```
Spotify confirmed  
Observed  
Inferred  
Provisional  
Unknown
```

Do not cover the main dashboard in provenance badges.

Source details should be available when requested.

71. Error Philosophy

Never silently discard data.

If an importer encounters something unknown:

```
preserve raw record  
record warning  
continue when safe
```

If unsafe:

```
abort transaction  
report exact reason
```

72. Initial Implementation Phases

Phase 0 — Specification

This document.

Freeze major terminology and architectural principles before implementation.

Phase 1 — Core Database

Implement:

- migrations;
 - import batches;
 - raw event storage;
 - content entities;
 - canonical plays;
 - provenance;
 - reconciliation tables.
-

Phase 2 — Extended Streaming History

Implement:

- file validation;
- parsing;
- raw storage;
- normalization;
- deduplication;
- basic canonical creation.

At the end of Phase 2, the system should already be able to import lifetime Spotify history.

Phase 3 — Historical Analytics

Implement:

- track statistics;
 - artist statistics;
 - album statistics;
 - daily/monthly/yearly totals;
 - history querying.
-

Phase 4 — Core Dashboard

Implement:

- Home;
- History;
- Tracks;

- Artists;
- Albums;
- Calendar;
- Search.

At this stage, the application should already be highly useful using only Extended Streaming History.

Phase 5 — Spotify Authentication and Current Playback

Implement:

- PKCE authentication;
 - token management;
 - current playback;
 - current device;
 - current context;
 - status/error handling.
-

Phase 6 — Reconciliation

Integrate:

- Recently Played where appropriate;
 - provisional play matching;
 - source matching;
 - match scoring;
 - reconciliation audit trail;
 - future Extended History reconciliation.
-

Phase 7 — Derived Telemetry Subsystem

Implement separately behind:

ENABLE_DERIVED_LISTENING_METRICS

The application must operate normally with the entire subsystem disabled.

Phase 8 — Advanced Analytics

Implement:

- comparison mode;
 - listening sessions;
 - discovery;
 - rediscovery;
 - trends;
 - coverage-aware metrics;
 - device analysis.
-

Phase 9 — Data Management

Implement:

- backups;
 - exports;
 - rebuild;
 - diagnostics;
 - database integrity tools;
 - import management.
-

73. Minimum V1 Acceptance Criteria

The first production-quality release should satisfy all of these:

1. A fresh installation works without an existing database.
2. Spotify Extended Streaming History can be imported.
3. Re-importing the same data does not duplicate history.
4. Overlapping exports can safely be imported.
5. Historical records remain available even when Spotify APIs are unavailable.
6. Every user-facing play is represented once.
7. Source provenance is preserved internally.
8. Unknown fields are not treated as zero.
9. UTC/local-time conversions are correct.
10. Track, artist, album, daily, monthly, yearly, and lifetime statistics work.
11. Search works.
12. Import history is visible.
13. Data coverage is visible.
14. Raw Spotify input is preserved.
15. Backups can be created.
16. Spotify API authentication is separated from historical imports.

17. The application works with:

```
ENABLE_DERIVED_LISTENING_METRICS=false
```

1. Switching the flag to `true` activates only the explicitly gated derived subsystem.
2. Derived information is never mislabeled as Spotify-authoritative.
3. A later Spotify export can reconcile earlier provisional records without double-counting.

74. Ultimate Target

The final program should function as a personal Spotify listening database where the user can ask the application, through its interface and analytics tools:

```
What am I listening to right now?  
  
What did I listen to today?  
  
What was my most listened track this week?  
  
How much Sabrina Carpenter did I listen to in August?  
  
When did I first hear this song?  
  
How has my listening to this artist changed over time?  
  
What songs have I recently started listening to heavily?  
  
What songs have I stopped listening to?  
  
What did I listen to at 1:34 AM on a particular date?  
  
Which tracks do I skip most?  
  
How much music did I listen to this year?  
  
What were my listening habits three years ago?  
  
How complete is the data behind this statistic?
```

The program should be able to answer these questions quickly from its own local database.

Its architecture should make the distinction between:

What Spotify definitively recorded
What the application directly observed
What the application inferred
What remains unknown

without forcing the user to think about those distinctions during normal everyday use.

The user's experience should simply be:

Listen on Spotify
↓
Open Tracker
↓
See everything

while the underlying system handles imports, provenance, reconciliation, duplicate detection, source authority, and data quality automatically.

Personal Spotify Listening Tracker V2

UI/UX Visual Design Specification - Incorporated Addendum

Specification version: 1.1

Status: Revised implementation specification - approved UI direction incorporated

Authority: Sections 75-110 below are normative. Where they conflict with the brief guidance in Sections 68-70, these sections take precedence.

75. Approved Visual Direction

The generated dashboard mockup approved during design review is the primary visual reference for the first implementation. It establishes the application's density, navigation model, restrained dark theme, data hierarchy, table treatment, album-art usage, chart proportions, and overall desktop-app character.

The reference is not a pixel-perfect contract for every literal number or example label shown in the mockup. It is a contract for the **visual language and composition**. Production data, accessibility requirements, responsive constraints, Spotify attribution rules, and the functional requirements elsewhere in this specification take precedence over placeholder content in the image.

Critical implementation rule: Codex or any other implementation agent must not independently redesign the UI. Design changes require an explicit specification or approved design revision.

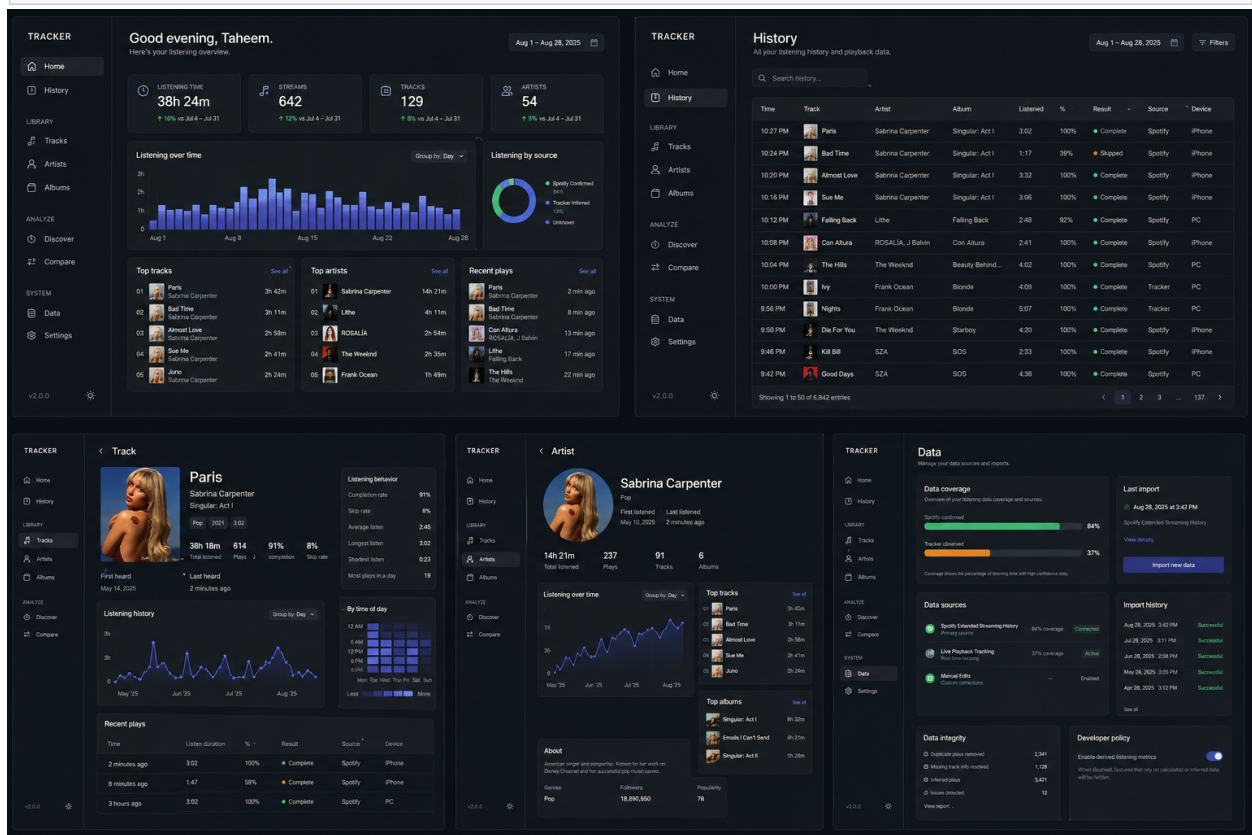


Figure 1 - Approved UI direction overview: Home, History, Track, Artist, and Data screens.

76. Visual Product Character

The application should feel like a purpose-built personal music analytics desktop application: quiet, dense, precise, and content-first. It should feel closer to a polished productivity/data product than a marketing website, a Spotify clone, or a generic dashboard template.

76.1 Desired qualities

- **Music-first:** album and artist artwork provide much of the visual personality and color.
- **Information-dense:** useful data is visible without requiring excessive scrolling or oversized cards.
- **Calm:** surfaces, borders, and typography support the data instead of competing with it.
- **Consistent:** the same spacing, typography, status semantics, chart grammar, and interaction patterns recur across pages.
- **Desktop-native in feel:** permanent sidebar, clear page headers, dense tables, predictable filters, and keyboard-friendly interactions.
- **Traceable:** advanced data provenance is available without contaminating normal screens with technical noise.

76.2 Explicitly prohibited visual patterns

- Glassmorphism, frosted panes, background blur, or translucent decorative overlays.
- Decorative gradients, aurora blobs, neon glows, bloom effects, or glowing borders unless a future approved design explicitly calls for them.
- Large rounded cards around every statistic or section.
- Oversized marketing-style headings and hero layouts.
- Pill-shaped containers used merely because they are fashionable; pills are reserved for tags, compact filters, statuses, and segmented controls.
- Emojis used as interface icons.
- Random accent colors or per-page color schemes.
- Decorative charts that do not answer a real analytical question.
- 3D charts, novelty gauges, speedometers, or donut charts for ordinary scalar metrics.
- Animation whose sole purpose is to make the interface look "dynamic".
- Stock shadow styling presented unchanged as the final product design.
- Cards nested inside cards without a genuine semantic grouping reason.

77. Application Shell and Sidebar

The application shell is persistent across normal pages. On desktop it consists of a fixed-width left sidebar and a fluid main content region. The sidebar is visually quieter than the content area.

77.1 Sidebar structure

Preferred desktop navigation groups:

HOME
Home
History

LIBRARY
Tracks
Artists
Albums

ANALYZE
Discover
Compare
Calendar

SYSTEM
Data
Settings

- Expanded width target: **176-192 px**.
- Collapsed desktop width target, if implemented: **60-68 px**.
- Navigation icons: approximately **16-18 px**.
- Navigation text: approximately **13-14 px**.
- Active navigation uses a subtle filled neutral surface plus stronger text/icon contrast; it should not use a bright full-width accent bar.
- Section labels are small, muted, uppercase or small-caps-like text. They should not compete with navigation items.
- Application version and a small settings/theme control may sit at the bottom of the sidebar.
- Sidebar scroll should be avoided at common laptop heights; lower-priority groups may collapse if necessary.

77.2 Main content frame

- Desktop page gutter: approximately **20-28 px**.
- Main content should remain fluid rather than being trapped in an unnecessarily narrow centered column.
- Dense analytics pages may use nearly the full available width.
- Major page sections are separated primarily through spacing, alignment, subtle borders, and localized surfaces rather than full-width decorative bands.

78. Typography System

Typography must carry much of the hierarchy. The implementation should use a clean interface sans-serif with high-quality numerals. The preferred family is **Geist Sans**, with **Geist Mono** available sparingly for diagnostic or tabular technical values. A metrically compatible fallback stack must be provided.

Role	Target size	Weight	Notes
Page title	20-24 px	600-650	Compact, never marketing-scale
Section heading	13-15 px	600	Often uppercase/small label style for dense sections

Role	Target size	Weight	Notes
Primary body/UI	13-14 px	400-500	Default application copy
Secondary/meta	11-12.5 px	400	Muted but readable
Metric value	24-32 px	500-650	Use only when the metric deserves prominence
Table text	12-13 px	400-500	Dense, aligned, scannable
Micro-label	10-11 px	500-600	Status, source, axis, compact metadata

- Use **tabular numerals** for changing values, times, durations, percentages, rankings, and table columns where alignment matters.
- Do not use monospace for the general interface.
- Artist, track, and album titles use ordinary casing; avoid artificial all-caps music metadata.
- Truncation must preserve important context via tooltip or expandable detail when space is constrained.

79. Color and Theme System

Dark mode is the primary reference theme. Light mode remains a product requirement, but it must be derived from the same semantic tokens rather than designed as a separate application.

79.1 Dark-theme semantic targets

Token	Reference value	Purpose
--bg-primary	#0B0F13	Main application background
--bg-sidebar	#0D1218	Sidebar background
--surface-1	#11171D	Localized cards/panels
--surface-2	#151C23	Hover/raised/selected neutral surface
--border-subtle	#222A33	Default separators and borders
--text-primary	#F3F5F7	Primary content
--text-secondary	#A8B0B8	Secondary metadata
--text-tertiary	#6F7983	Tertiary labels
--accent	#6877F3	Primary selection/control accent
--success	#48C986	Confirmed/success when semantics require it
--warning	#E3A04B	Warning/provisional when semantics require it
--danger	#E46B6B	Errors/destructive states

79.2 Color usage rules

- The accent color is for interaction, selection, chart focus, links, and keyboard focus. It must not flood large decorative backgrounds.
- Album artwork and artist imagery are the richest visual color source in normal use.
- Green indicates success/confirmed only when that semantic distinction is actually useful; it is not a Spotify-brand wallpaper color.
- Muted neutral grays should dominate surfaces and separators.

- Text and controls must maintain accessible contrast. Never preserve a mockup color if it fails accessibility.

79.3 Light theme

Light mode should use a near-white canvas, slightly differentiated neutral surfaces, dark text, and the same accent. It must preserve hierarchy and density; light mode must not become a radically different card-heavy design.

80. Spacing, Density, Geometry, and Elevation

- Base spacing unit: **4 px**. Prefer multiples of 4 for component spacing.
- Common gaps: 4, 8, 12, 16, 20, 24, 32 px. New arbitrary gaps require justification.
- Default control height: approximately **32-36 px** desktop; touch targets expand appropriately on narrow/touch layouts.
- Dense table row height: approximately **38-44 px**.
- Default panel radius: **6 px**. Large content panels may use **8 px**. Avoid 16-24 px "bubble" cards.
- Small artwork radius: **4 px**; larger artwork may use **8 px** where Spotify display guidance permits.
- Use subtle 1 px borders. Shadows are rare and reserved for overlays such as dropdowns/dialogs; static panels should generally not float with dramatic shadows.
- Section hierarchy should first be achieved through spacing and typography, not boxes.

81. Surface and Card Rules

A surface exists only when it communicates grouping, state, or interaction. The application must not wrap every section in a card simply because the component library makes cards easy to generate.

81.1 Appropriate panel uses

- Data coverage, import status, and integrity summaries.
- A chart with associated controls that must read as one analytical unit.
- A compact metric group with a shared comparison period.
- Developer policy controls and other discrete settings groups.
- Modal or popover content.

81.2 Prefer flat structure for

- Ranked top-track and top-artist lists.
- History tables.
- Metadata labels under a track/artist header.
- Long analytical sections separated cleanly with headings and rules.
- Navigation and secondary page controls.

82. Artwork and Spotify Content Presentation

- Use authentic album/artist imagery where available. Artwork should provide visual recognition, not merely decoration.
- Do not blur, distort, recolor, overlay text onto, or aggressively crop Spotify-provided artwork where Spotify display rules prohibit such treatment.
- Reference sizes: 32 px dense table rows; 40 px standard lists; 48-56 px prominent recent-play rows; 160-240 px detail-page hero artwork depending on viewport.
- If artwork is missing, use a restrained neutral placeholder with an icon or initials. Do not invent AI-generated cover art.
- Spotify attribution/linking requirements must be implemented wherever Spotify metadata/artwork requires them.

83. Iconography

- Preferred icon family: **Lucide** or another single consistent stroke icon family.
- Typical size: 16-18 px; 20 px only for higher-prominence controls.
- Icons are not placed beside every label by default.
- Icon-only buttons require accessible names/tooltips.
- Do not mix filled, outlined, emoji, and multiple vendor icon sets without explicit design approval.

84. Page Header and Global Controls

- Page title sits at top-left of the content area and remains compact.
- Optional supporting sentence may appear beneath title in smaller muted text.
- Global date range belongs at top-right on analytics pages where it affects most of the page.
- Secondary controls such as Filters may sit adjacent to the date selector.
- Do not create a giant hero header for ordinary application pages.
- The date range component must clearly state the effective period and support custom ranges.

85. Metric Presentation

Metrics should be treated as information, not decorative tiles. The mockup permits compact metric panels on Home because the group forms a useful top-level summary; the same pattern should not be repeated mechanically on every page.

- A metric consists of label, value, optional comparison, and optional small icon.
- Comparison text must state the comparison basis or make it evident through the current date filter.
- Positive/negative color is used only when the direction has an actual positive/negative meaning. More listening time is not inherently "good" and must not automatically appear green.
- When a percentage is informational rather than value-judged, use neutral or accent styling.
- Metrics derived from incomplete coverage should expose coverage in detail/tooltip rather than falsely presenting universal certainty.

86. Charts and Data Visualization Grammar

86.1 Permitted primary chart types

- Line charts for time-series trends.
- Vertical/horizontal bar charts for rankings and discrete period comparisons.
- Calendar/heatmap grids for hour/day listening density.
- Timelines for coverage and event chronology.
- 100% or segmented bars for source/coverage composition.
- Donut charts only for a small number of true part-to-whole categories where a table or bar would be less clear.

86.2 Chart behavior

- Every chart has an explicit analytical question and an accompanying numeric summary or accessible data representation.
- Grid lines are subtle and sparse.
- Axis labels use secondary/tertiary text colors and remain legible.
- Hover/tap tooltips show exact values, date/time, and series identity.
- Selection persists where useful rather than disappearing immediately after pointer movement.
- Comparison series use distinguishable line/bar styles and semantic series colors, not a rainbow palette.
- Unknown/insufficient data uses gaps, hatching, dashed lines, or explicit labels rather than fabricated zeros.
- Animations are brief and must not make the chart harder to read.

86.3 Source/coverage visualization

The Data page may use distinct semantic colors to visualize Spotify-confirmed, tracker-observed, inferred, and unknown coverage. This palette is specific to provenance and must remain consistent throughout the application.

87. Tables and Dense Lists

History and other large datasets are table-first. Do not render thousands of plays as oversized individual cards.

- Header row remains visually distinct but restrained.
- Row height target: 38-44 px desktop.
- Artwork can appear inline with track text at approximately 28-34 px.
- Numeric columns align right where helpful; duration and percentage use tabular numerals.
- Primary text is brighter; album/device/source metadata is secondary.
- Rows support click/tap to open detail. Hover indicates interactivity but is not the only cue.

- Sorting indicators appear only on sortable columns.
- Pagination or virtualized scrolling may be used; loading thousands of DOM rows at once is not acceptable.
- Column visibility may adapt at narrower widths. Essential identity/time columns survive longest.
- Empty cells that mean unknown must not silently display 0.

88. Home Page - Detailed Layout



Figure 2 - Approved Home-page direction.

Home is an overview, not an exhaustive report. It should answer: *what have I been listening to, how much, and what is happening recently?*

88.1 Desktop composition

- Top row: compact greeting/context at left; current global period selector at right.
- Summary row: approximately four metrics on wide desktop: Listening Time, Streams/Plays, Tracks, Artists. The exact metric set may adapt to the active analytics definition.
- Primary analytical row: a wide Listening Over Time chart occupying roughly two-thirds of the row; a narrower source/coverage or contextual summary occupies the remainder.
- Lower row: Top Tracks, Top Artists, and Recent Plays in three aligned columns on wide screens.
- Now Playing may appear as a compact module when active, but it must not displace the entire analytics page.

88.2 Home interactions

- Clicking a top track/artist navigates to its detail page.
- Clicking Recent Plays opens play detail or History at that event.
- Chart grouping can switch among sensible periods such as hour/day/week/month depending on range.

- Date-range changes update all Home analytics consistently.

89. History Page - Detailed Layout

Time	Track	Artist	Album	Listened	%	Result	Source	Device
10:27 PM	Paris	Sabrina Carpenter	Singular: Act I	3:02	100%	Complete	Spotify	iPhone
10:24 PM	Bad Time	Sabrina Carpenter	Singular: Act I	1:17	39%	Skipped	Spotify	iPhone
10:20 PM	Almost Love	Sabrina Carpenter	Singular: Act I	3:32	100%	Complete	Spotify	iPhone
10:16 PM	Sue Me	Sabrina Carpenter	Singular: Act I	3:06	100%	Complete	Spotify	iPhone
10:12 PM	Falling Back	Lithe	Falling Back	2:48	92%	Complete	Spotify	PC
10:08 PM	Con Altura	ROSALÍA, J Balvin	Con Altura	2:41	100%	Complete	Spotify	iPhone
10:04 PM	The Hills	The Weeknd	Beauty Behind...	4:02	100%	Complete	Spotify	PC
10:00 PM	Ivy	Frank Ocean	Blonde	4:09	100%	Complete	Tracker	PC
9:56 PM	Nights	Frank Ocean	Blonde	5:07	100%	Complete	Tracker	PC
9:50 PM	Die For You	The Weeknd	Starboy	4:20	100%	Complete	Spotify	iPhone
9:46 PM	Kill Bill	SZA	SOS	2:33	100%	Complete	Spotify	iPhone
9:42 PM	Good Days	SZA	SOS	4:38	100%	Complete	Spotify	PC

Figure 3 - Approved History-page direction.

- Header: History title, short explanation, date range, Filters control.
- Search field directly above the table for fast narrowing by track/artist/album.
- Primary desktop columns: Time, Track, Artist, Album, Listened, Completion %, Result, Source, Device. Columns may vary according to data availability.
- Result should distinguish Completed/Skipped/Partial/Unknown without overusing colored badges.
- Source may show simplified labels such as Spotify, Tracker, or Combined; detailed provenance belongs in the row detail view.
- Footer provides result count and pagination/virtualization status.
- Filters support date/time, content identity, platform/device, shuffle, skip, offline/private where data exists, source coverage, and reconciliation status.
- Selecting a row opens a side panel or dedicated detail page; choose one interaction and use it consistently.

90. Track Detail Page - Detailed Layout

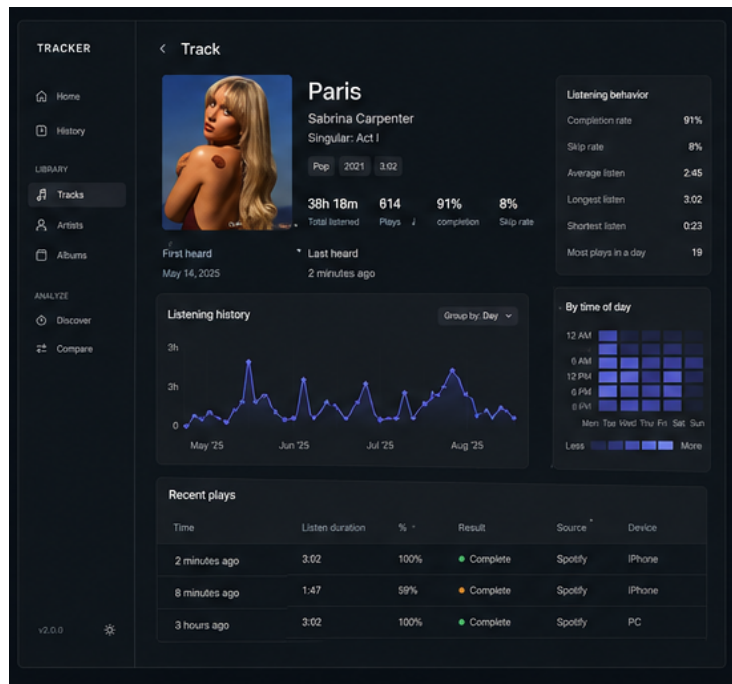


Figure 4 - Approved Track-detail direction.

- Hero region: artwork at left; track title, artist, album, compact tags/metadata, first-heard and last-heard information.
- Primary metrics near hero: total listened, plays, completion, skip rate when eligible.
- Main body: Listening History line chart and a time-of-day/day-of-week heatmap or equivalent behavior visualization.
- Right-side or adjacent behavior panel may summarize completion rate, skip rate, average listen, longest/shortest eligible listen, and most plays in one day.
- Recent Plays table appears beneath analytics.
- Unknown or ineligible metrics are omitted or labeled unavailable; they are never rendered as 0 merely to fill a layout.

91. Artist Detail Page - Detailed Layout

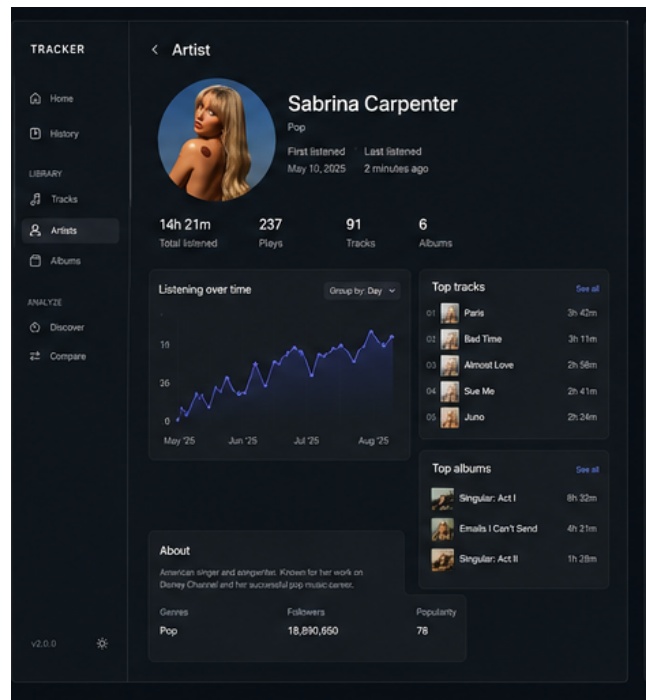


Figure 5 - Approved Artist-detail direction.

- Header includes artist image, name, optional genre metadata where permitted, first/last heard, and lifetime metrics.
- Main chart shows listening over time with controllable grouping.
- Top Tracks and Top Albums appear as ranked compact lists rather than separate oversized cards.
- Additional metadata such as genres/popularity may be presented only if sourced reliably and permitted; it should be visually subordinate to personal listening data.
- Artist pages should make long-term trend changes immediately visible.

92. Album Detail Page - Detailed Layout

- Album artwork, title, primary artists, release metadata, first/last listen, total album listening, and total plays form the header region.
- Track-by-track ranking should use a dense table/list with play count and listening time.
- Album listening timeline appears as a line or bar chart.
- Potential full-album sessions, when implemented, must be clearly labeled as detected/inferred according to data authority.
- Do not imitate Spotify's album playback page; this is an analytics page, not a player surface.

93. Data Page - Detailed Layout

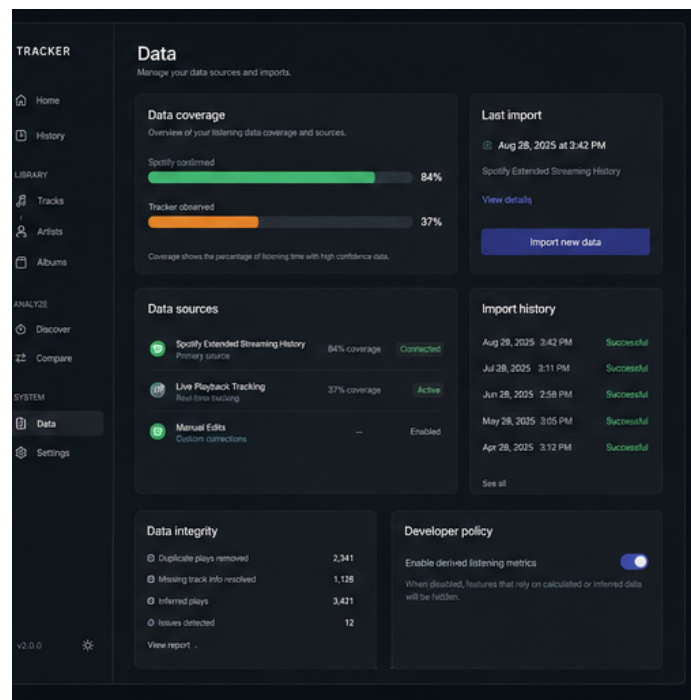


Figure 6 - Approved Data-page direction.

The Data page is the primary technical surface. It may be denser and more explicit than ordinary music pages, but it must still look like the same application.

- Data Coverage panel showing Spotify-confirmed and tracker-observed coverage with percentages/timeline.
- Last Import panel with timestamp, source, status, and Import New Data action.
- Data Sources list with source name, coverage, and connection/activity state.
- Import History table with date, source/package, record counts where useful, and status.
- Data Integrity panel with duplicate plays removed, ambiguous matches, inferred plays, issues detected, database integrity, and links to reports.
- Developer Policy panel containing the derived-listening-metrics feature gate and warning.

94. Import Flow UI

Importing Spotify data should feel deliberate and safe. The normal workflow is a short multi-step flow rather than a single opaque file picker.

94.1 Import steps

1. **Select files/package** - drag/drop and browse are both supported.
2. **Validate** - identify package type, files, date coverage, event count, and warnings.
3. **Preview** - show expected new, duplicate, overlapping, conflicting, and unsupported records where determinable.
4. **Import** - show progress with cancellability only if cancellation is transactionally safe.
5. **Result** - summarize exact outcomes and links to conflicts/reconciliation details.

94.2 Import visual rules

- The primary Import action is prominent but not oversized.

- Warnings use concise inline callouts; do not use dramatic full-screen red states for recoverable issues.
- Record counts use tabular numerals.
- The user must never wonder whether importing the same package again will duplicate history; the UI should explicitly state that duplicates are safely detected.

95. Settings Page

- Group settings by General, Spotify Connection, Tracking, Data, Appearance, Developer/Advanced.
- Use native-feeling rows with label, description, and control. Avoid presenting each setting as an independent floating card.
- Destructive actions are visually separated from ordinary preferences.
- Advanced settings are collapsible or placed in a dedicated section to protect normal users from implementation detail.

95.1 Derived Listening Metrics control

The UI representation of `ENABLE_DERIVED_LISTENING_METRICS` must accurately reflect the architectural feature gate defined in Section 8.

- Default control state: **Off**.
- Label: **Enable derived listening metrics**.
- Supporting copy: explains that enabling activates live-derived/inferred listening analytics and does not itself establish Spotify-policy compliance.
- Turning the setting on requires an explicit confirmation dialog summarizing the consequence.
- Turning it off stops gated workers and metric production according to Section 8; existing raw evidence is not secretly rewritten.
- The switch state must reflect the effective runtime configuration rather than a disconnected decorative preference.

96. Now Playing

Now Playing is transient operational information and should visually remain distinct from finalized historical analytics.

- Show artwork, track, artist, album, progress, playing/paused state, device, context, shuffle, and repeat where available.
- Progress is a simple horizontal timeline, not a decorative waveform.
- Current playback may appear as a compact Home module or a persistent mini-panel, but the initial implementation should avoid a full Spotify-player clone.
- No active device and Nothing playing are normal states, not errors.
- When derived metrics are disabled, Now Playing still works using direct permitted state display without silently turning observations into derived history.

97. Search, Filters, and Controls

- Search fields use clear placeholder examples and support keyboard focus.
- Filters open a compact popover/sheet containing grouped filter controls; applied filters are summarized visibly.
- Use pills/chips only for active filter tokens or compact tags, not as default containers for ordinary controls.
- Date range selector supports standard presets and custom start/end dates.
- Segmented controls are appropriate for a small mutually exclusive set such as Day/Week/Month grouping.
- Dropdowns and popovers use consistent width, border, radius, and elevation.
- All controls must work by click/tap; hover can enhance but cannot be required.

98. Provenance and Data Status UI

The main UI should not be buried under provenance labels, but advanced/detail views must expose data authority clearly.

State	Meaning	Normal presentation
Spotify confirmed	Direct authoritative historical field	Usually implicit; shown in details
Observed	Direct tracker observation	Advanced/details
Inferred	Derived by tracker logic	Advanced/details; warning semantics when relevant
Provisional	Not yet reconciled with later authoritative export	Subtle indicator when it materially affects confidence
Unknown	No reliable value	Omit or explicitly show unavailable; never 0

99. Loading, Empty, Error, and Offline States

99.1 Loading

- Prefer skeleton rows/sections that preserve layout geometry.
- Avoid global spinners when only one panel is loading.
- Long imports/rebuilds show progress, current stage, and meaningful record counts where available.

99.2 Empty states

- State what is missing and provide the next useful action.
- Do not fill empty states with decorative AI illustrations.
- Example: "No listening history imported yet" + Import Spotify Data action.

99.3 Error states

- Errors state what failed, what data remains safe, and what the user can do next.
- Rate limiting should show a non-alarming retry state.
- Spotify disconnected/authentication expired states provide a reconnect action.
- Historical analytics continue to function when Spotify is offline if the local database is available.

100. Motion and Interaction

- Default UI transitions: approximately 120-180 ms for hover/focus/popover state and 180-240 ms for larger layout transitions.
- Use motion to explain continuity: expanding row detail, opening a popover, changing a selected chart range.
- Do not animate every card on page load.
- Do not bounce metric values, rotate album art, create cursor-following effects, or use perpetual ambient motion.
- Respect `prefers-reduced-motion`.
- If Motion is used, it should wrap approved interaction patterns rather than become a design generator.

101. Accessibility Requirements

- All functionality must be keyboard accessible.
- Visible focus indicators must use the semantic accent and meet contrast requirements.
- Interactive targets require accessible names.
- Charts need accessible summaries/data alternatives.
- Color may not be the sole carrier of status meaning; include text/icon/shape distinctions.
- Text must preserve readable contrast in both themes.
- Table headers must be programmatically associated with columns where applicable.
- Touch layouts should provide approximately 44 px targets even when desktop visual density is tighter.
- Reduced-motion preference must be honored.

102. Responsive Behavior

Desktop is the primary target, but the application must degrade intentionally rather than merely shrinking the desktop layout.

Viewport	Expected behavior
≥ 1280 px	Full sidebar; multi-column Home; wide History table; side-by-side detail analytics.
960-1279 px	Full or compact sidebar; Home lower sections reduce columns; secondary chart panels stack when necessary.
720-959 px	Sidebar may collapse; two-column layouts become one/two-column hybrids; History hides lower-priority columns.
< 720 px	Drawer navigation; single-column analytics; horizontal table scroll or mobile history rows; controls become touch-

- No page-level horizontal overflow at narrow widths except deliberate local scrollers such as wide data tables.
- Album/artist detail headers stack cleanly; artwork does not consume most of the screen.
- Charts remain legible and may reduce axis labeling rather than simply scaling text to unreadable sizes.
- The mobile layout remains an analytics application, not a completely separate visual language.

103. Component System

The frontend should define reusable primitives before feature pages are mass-produced. This is a key anti-slop requirement.

Core components should include, at minimum:

```
Button, IconButton, TextField, SearchField, Select, Dropdown, Tabs, SegmentedControl,
Checkbox, Switch, Tooltip, Popover, Dialog, SidebarItem, PageHeader, Toolbar, FilterBar,
Metric, Artwork, TrackRow, ArtistRow, AlbumRow, PlayRow, DataTable,
Pagination/VirtualList, EmptyState, ErrorState, LineChart, BarChart, Heatmap, Timeline,
StatusIndicator, DataSourceIndicator, ImportSummary, CoverageBar.
```

- A component is styled once through tokens and variants.
- Feature pages compose existing components rather than creating slightly different local clones.
- New variants require a real semantic difference, not convenience.

104. Design Tokens

Design values must be centralized. Implementation agents may not invent arbitrary colors, radii, shadows, spacing values, or type scales directly in page code.

```
:root { --space-1: 4px; --space-2: 8px; --space-3: 12px; --space-4: 16px; --space-5:
20px; --space-6: 24px; --radius-sm: 4px; --radius-md: 6px; --radius-lg: 8px;
--control-h-sm: 32px; --control-h-md: 36px; }
```

Exact token implementation may use CSS variables, Tailwind theme configuration, or an equivalent centralized system. The semantic intent is mandatory even if names differ slightly.

105. Frontend Implementation Stack

Preferred V1 frontend stack:

- **React + TypeScript** for application UI.
- **Vite** for the local frontend build unless another approved integration is demonstrably simpler.
- **Tailwind CSS** or an equivalent token-driven utility layer for predictable implementation.
- **Radix primitives / shadcn component source** may be used for accessible behavior, but stock visual styling must be replaced by this design system.
- **Lucide** for iconography.
- **Motion** only for approved interaction transitions.
- **Recharts** or an equivalently mature chart library may be used behind tracker-specific chart wrappers so that styling and semantics remain centralized.

The Python backend remains authoritative for data access, import, reconciliation, and analytics. The frontend consumes a local application API and must not duplicate canonical analytics logic in ad hoc client code.

106. Codex / Implementation-Agent Visual Rules

The following block should be included in repository-level implementation instructions:

VISUAL IMPLEMENTATION RULES Do not independently redesign the application. Follow the approved mockups, design tokens, and component system. Do not add unless explicitly specified: - gradients or decorative color washes - glassmorphism or blur - decorative glows - large shadows - decorative floating shapes - excessive border radii - unnecessary cards - pill containers without semantic purpose - emojis as UI icons - arbitrary accent colors - marketing-style hero headings - decorative charts - animations without interaction purpose - new one-off design tokens Prefer: - typography - spacing - alignment - subtle borders - restrained surfaces - dense tables/lists - album artwork - meaningful charts - existing shared components

107. Visual QA Workflow

UI implementation is not complete when the page merely compiles. Each key page must be visually compared against the approved reference and reviewed at representative viewport sizes.

107.1 Required review sequence

1. Implement design tokens and base components.
 2. Implement one reference page.
 3. Capture screenshot at approved desktop viewport.
 4. Compare hierarchy, spacing, density, type scale, alignment, surface use, and chart treatment against the approved design.
 5. Correct deviations.
 6. Only then reuse the established components for additional pages.
- Do not solve a mismatch by redesigning the reference.
 - Do not add new decorative elements to make sparse development data look more "finished".
 - Visual regression screenshots should be retained for core pages after the design stabilizes.
 - Dark and light themes must both undergo visual QA.

108. Key-Screen Design Order

The initial UI should be implemented and approved in this order because these screens collectively exercise almost every visual primitive:

1. Home 2. History 3. Track Detail 4. Artist Detail 5. Data / Imports 6. Settings

Album, Calendar, Compare, Discover, and other analytical pages should then inherit the established system. They should not start new visual languages.

109. UI Acceptance Criteria

The UI portion of the application is accepted only when all of the following are true:

- The implementation clearly resembles the approved visual reference in density, hierarchy, navigation, and restraint.
- Sidebar and page shell are consistent across all normal screens.
- Design tokens centrally define colors, spacing, radii, type sizes, and motion durations.
- No page uses unapproved gradients, glassmorphism, decorative glow, or arbitrary one-off visual effects.
- History is table/list based and remains usable with large datasets.

- Home provides a useful overview without becoming a grid of redundant cards.
- Track and Artist pages combine artwork, core metrics, trends, and dense supporting data without mimicking Spotify player pages.
- Data/Imports makes provenance, coverage, import status, and integrity understandable.
- The derived-listening-metrics control accurately reflects and controls the Section 8 feature gate.
- Unknown data is not visually represented as zero.
- Charts follow one consistent visual grammar and expose exact values through accessible interaction.
- Album/artist artwork is used respectfully and does not become decorative blurred wallpaper.
- All major interactions work by keyboard and pointer/touch.
- Both dark and light themes meet contrast/accessibility requirements.
- Responsive layouts remain usable at tablet and mobile widths without page-level clipping.
- Loading, empty, disconnected, rate-limited, and error states are designed rather than left as raw developer messages.
- The frontend remains fully usable when `ENABLE_DERIVED_LISTENING_METRICS=false`.
- Implementation agents do not silently introduce new visual patterns outside the approved system.

110. Visual End State

The target is not to make the tracker look "AI-designed." The target is for it to look like a coherent, intentionally designed application whose visual decisions are boring in the best sense: repeatable, restrained, legible, and obviously serving the music and the data.

The user should open the application and immediately recognize three things: **what they are listening to, what their listening history looks like, and where to investigate further**. The interface should make those answers feel effortless while keeping the complex provenance/reconciliation machinery available beneath the surface.

Implementation mantra: Music supplies the personality. Data supplies the substance. The interface supplies structure - and then gets out of the way.